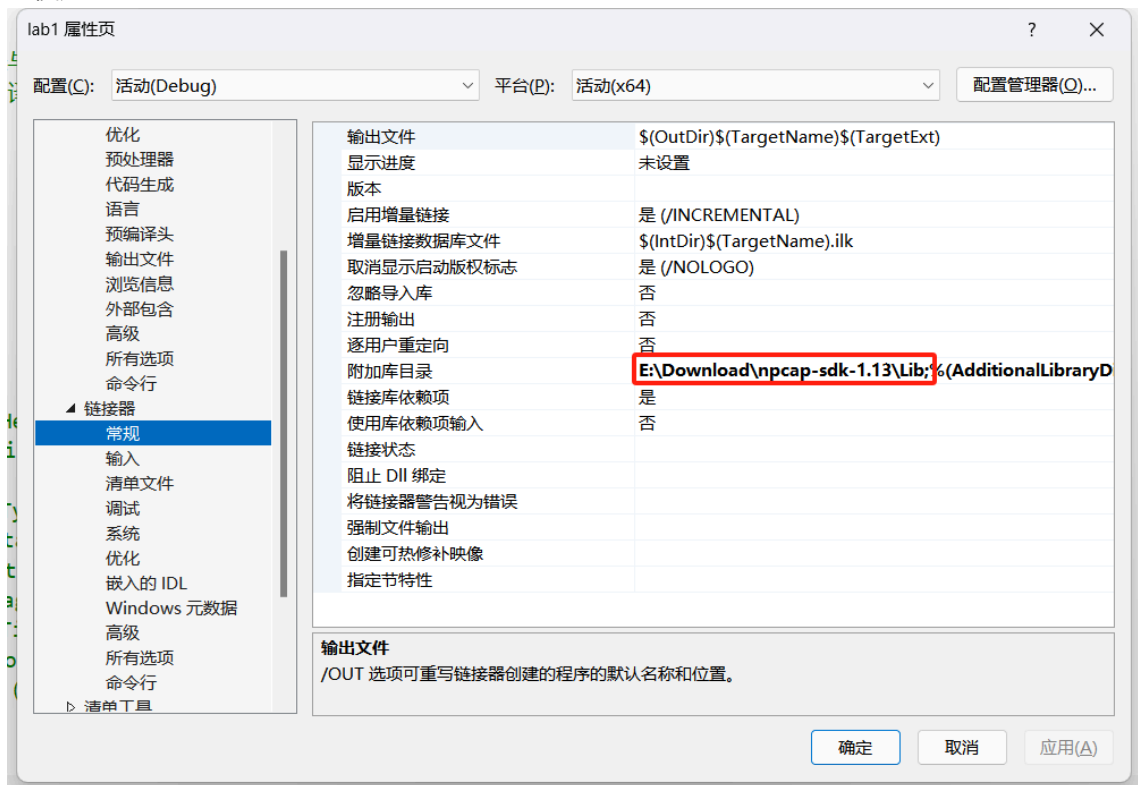
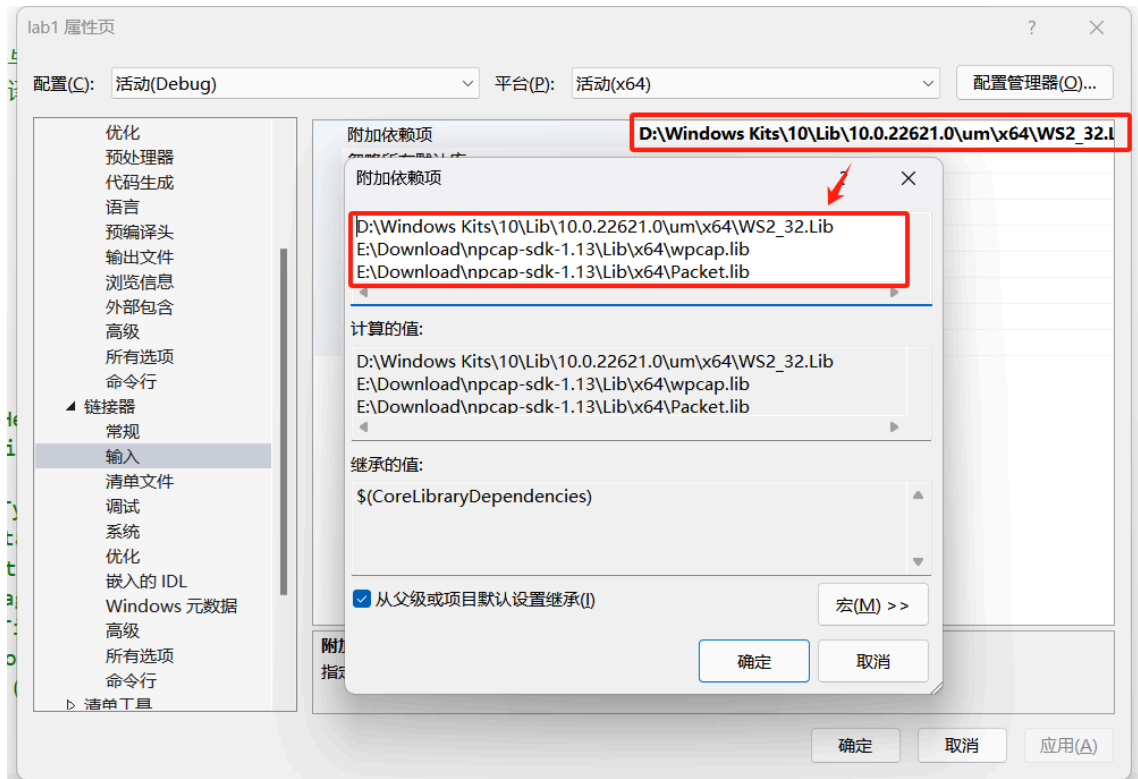


2. 添加库文件目录：在生成可执行的应用程序时，生成程序需要知道库文件所在的磁盘目录，供连接器使用。



3. 添加链接时使用的库文件：在基于 Npcap 应用程序生成过程中，需要链接 `wpcap.lib` 和 `Packet.lib` 库文件。

注意：书上提供的只需要添加 `wpcap.lib` 和 `Packet.lib` 库文件，但在实际的实现过程中会出现找不到 `pcap.h` 库的情况，所以需要添加 `windows kits` 中的 `ws2_32.Lib` 库文件。



不添加 `ws2_32.Lib` 库文件的报错信息：

LNK201: 无法解析的外部符号 <code>pcap_open_live</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK201: 无法解析的外部符号 <code>pcap_loop</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK201: 无法解析的外部符号 <code>pcap_freealldevs</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK201: 无法解析的外部符号 <code>pcap_freealldevs</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK427: 源计算机类型 'x86' 与目标计算机类型 'x64' 冲突	lab1	Packet.lib	1
LNK427: 源计算机类型 'x86' 与目标计算机类型 'x64' 冲突	lab1	wpcap.lib	1

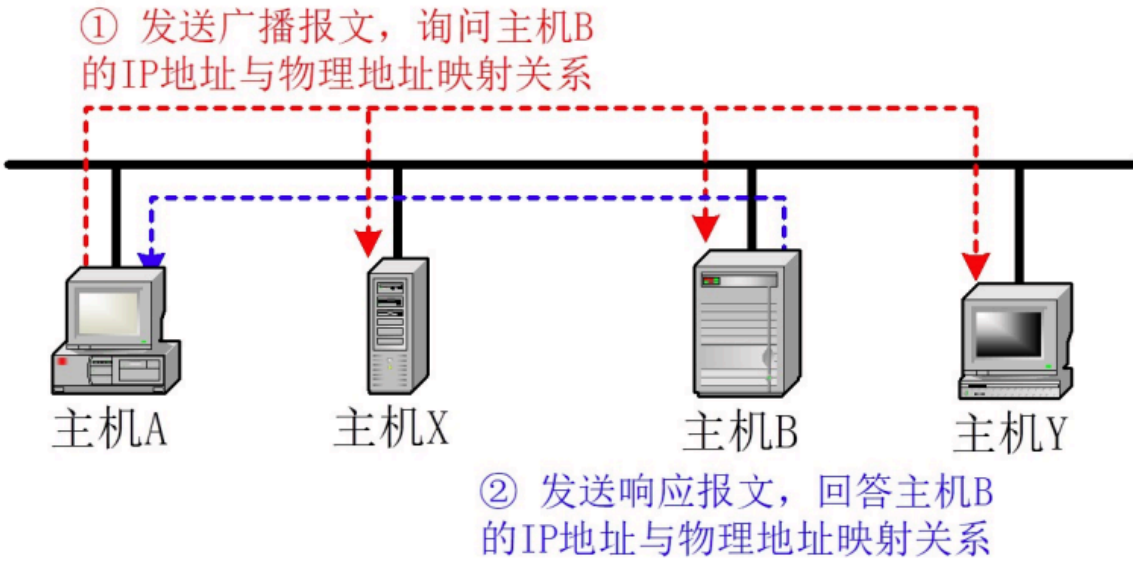
实现思路

本次实验使用 ARP 报文实现。

ARP

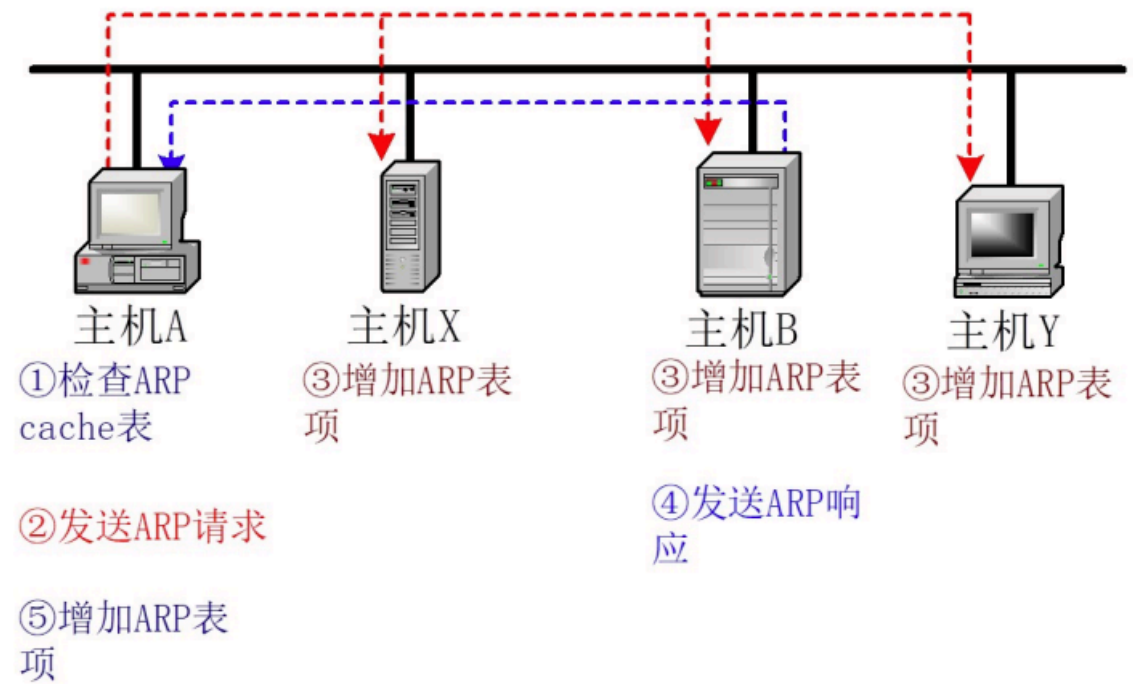
ARP 是地址解析协议。

ARP 的基本思想：



随着技术的发展改进，为加快 ARP 的寻址解析速度，主机采用高速缓存技术。即使用 Cache 保存已知的 ARP 表项，主机获得其他 IP 地址与物理地址映射关系后存入该 Cache。发送时先检索 Cache，若找不到再利用 ARP 解析，同时并利用计时器保证 Cache 中 ARP 表项的时效性。

改进后的 ARP 完整过程：



ARP 报文格式：

0	15	16	31
硬件类型		协议类型	
硬件地址长度	协议地址长度	操作	
源MAC地址（0-3）			
源MAC地址（4-5）		源IP地址（0-1）	
源IP地址（2-3）		目的MAC地址（0-1）	
目的MAC地址（2-5）			
目的IP地址（0-3）			

硬件类型: 以太网接口类型为 1

协议类型: IP 协议类型为 0800h

操作: ARP 请求为 1, ARP 应答为 2

硬件地址长度: MAC 地址长度为 6B

协议地址长度: IP 地址长度为 4B

源 MAC 地址: 发送方的 MAC 地址

源 IP 地址: 发送方的 IP 地址

目的 MAC 地址: ARP 请求中该字段没有意义; ARP 响应中为接收方的 MAC 地址

目的 IP 地址: ARP 请求中为请求解析的IP地址; ARP 响应中为接收方的 IP 地址

实现步骤

概览: 1. 定义相关变量 2. IP地址与MAC地址的对应: 发送广播, 利用发送并接收ARP报文来获取IP和MAC对应关系

1. 定义以太网首部和 ARP 报文结构包。以太网首部的结构体有三个成员变量: 一个长度为 6 的无符号字符数组 `dest_mac`, 用于存储目的 MAC 地址; 一个长度为 6 的无符号字符数组 `src_mac`, 用于存储源 MAC 地址; 一个无符号短整型 `ether_type`, 用于表示以太网帧的类型。 `ArpHeader` 的结构体, 用于表示 ARP 的头部结构。成员变量含义在代码后注释中。

```
// 以太网首部
typedef struct EthernetHeader {
    u_char dest_mac[6];
    u_char src_mac[6];
    u_short ether_type;
}EthernetHeader;

// ARP结构
typedef struct ArpHeader {
    unsigned short HdType;    // 硬件类型
    unsigned short ProType;   // 协议类型
```

```

    unsigned char HdSize;      // 硬件地址长度
    unsigned char ProSize;     // 协议地址长度
    unsigned short OP;         // ARP请求1 ARP应答2 RARP请求3 RARP应答4
    u_char SrcMac[6];
    u_char SrcIp[4];
    u_char DestMac[6];
    u_char DestIp[4];
}ArpHeader;

```

2. 构建 ARP 请求包。先初始化以太网头部，由于 ARP 请求需要广播到整个网络，所以将目的 MAC 地址设置为广播地址（FF:FF:FF:FF:FF:FF）。如果传入 `src_mac` 不为空，使用提供的源 MAC 地址填充以太网头部和 ARP 头部中的源 MAC 字段；如果 `src_mac` 为空，使用占位符 MAC 地址（0F:0F:0F:0F:0F:0F）代替。

接下来初始化 ARP 头部，因为目标设备的 MAC 地址未知，设置 ARP 目的 MAC 地址为广播地址（FF:FF:FF:FF:FF:FF），并将源 IP 地址（`src_ip`）和目标 IP 地址（`dest_ip`）填充到 ARP 头部。

设置 ARP 协议字段，指定以太网协议类型为 ARP（0x0806），硬件类型为以太网（1），协议类型为 IPv4（0x0800），硬件地址长度 6 字节 MAC 地址和协议地址长度 4 字节 IP 地址，设置 ARP 操作为请求（宏定义中设定 ARP 请求为 1）。

构造完整数据包，`memset(sendbuf, 0, 42)` 将发送缓冲区清零，避免遗留数据干扰，

`memcpy(sendbuf, eh, sizeof(*eh))` 将以太网头部拷贝到缓冲区起始位置，`memcpy(sendbuf + sizeof(*eh), ah, sizeof(*ah))` 将 ARP 头部紧接着以太网头部拷贝到缓冲区。

最终，`sendbuf` 中包含一个完整的 ARP 请求包，可直接通过网络发送。

```

void BuildRequest(EthernetHeader* eh, ArpHeader* ah, u_char* sendbuf, const
u_char* src_mac, const u_char* src_ip, const u_char* dest_ip) {
    memset(eh->dest_mac, 0xff, 6);
    if (src_mac != nullptr) {
        memcpy(eh->src_mac, src_mac, 6);
        memcpy(ah->SrcMac, src_mac, 6);
    }
    else {
        memset(eh->src_mac, 0x0f, 6);
        memset(ah->SrcMac, 0x0f, 6);
    }

    memset(ah->DestMac, 0xff, 6);
    memcpy(ah->SrcIp, src_ip, 4);
    memcpy(ah->DestIp, dest_ip, 4);

    eh->ether_type = htons(ETH_ARP);
    ah->HdType = htons(ARP_HARDWARE);
    ah->ProType = htons(ETH_IP);
    ah->HdSize = 6;
    ah->ProSize = 4;
    ah->OP = htons(ARP_REQUEST);

    memset(sendbuf, 0, 42);
    memcpy(sendbuf, eh, sizeof(*eh));
    memcpy(sendbuf + sizeof(*eh), ah, sizeof(*ah));
}

```

3. 报文抓取及判断。CapPacket 函数的作用是捕获网络中的 ARP 包，并检查其中是否有匹配的 IP 地址和 MAC 地址的包。如果找到匹配的包，可以将其源 IP 和源 MAC 地址保存到指定的变量中。
- 初始化捕获循环，Packet_Header 用于存储捕获数据包的元信息，Packet_Data 指向捕获到的数据包内容，errbuf 用于存储错误信息。
- 循环捕获数据包，调用 pcap_next_ex 函数捕获一个数据包：返回值为 -1，表示捕获错误；返回值为 0，表示超时，但没有捕获到包；返回值为 1，表示成功捕获一个包。
- 处理捕获到的数据包，将数据包内容转换为 ArpPacket 类型，调用 PrintHeader 函数打印数据包的头部信息。
- 解析 ARP 协议部分，将捕获到的数据包中的 ARP 协议部分提取出来，调用 Compare 函数，分别检查 ARP 包中的目标 IP 地址是否与 src_ip 匹配、源 IP 地址是否与 dest_ip 匹配。匹配成功时后，打印捕获到源 IP 地址和源 MAC 地址。

```
bool CapPacket(pcap_t* handle, const u_char* src_ip, const u_char* dest_ip,
u_char* sendIp = nullptr, u_char* sendMac = nullptr) {
    pcap_pkthdr* Packet_Header;
    const u_char* Packet_Data;
    char errbuf[256];

    while (true) {
        int n = pcap_next_ex(handle, &Packet_Header, &Packet_Data);
        if (n == -1) {
            cout << "Error: " << errbuf << endl;
            return false;
        }
        else if (n != 0) {
            ArpPacket* IPPacket = (ArpPacket*)Packet_Data;
            PrintHeader(Packet_Data);
            struct ArpHeader* arp_protocol = (struct ArpHeader*)(Packet_Data +
sizeof(struct EthernetHeader));

            if (Compare(arp_protocol->DestIp, src_ip) && Compare(arp_protocol-
>SrcIp, dest_ip)) {
                cout << "Captured ARP packet with matching IP and MAC address:"
<< endl;

                cout << "IP: "; PrintIp(arp_protocol->SrcIp);
                cout << "MAC: "; PrintMac(arp_protocol->SrcMac);

                if (sendIp != nullptr && sendMac != nullptr) {
                    // arp_protocol->SrcIp -> sendIp
                    for (int j = 0; j < 4; ++j)
                        sendIp[j] = arp_protocol->SrcIp[j];
                    // arp_protocol->SrcMac -> sendMac
                    for (int j = 0; j < 6; ++j)
                        sendMac[j] = arp_protocol->SrcMac[j];
                }
                break;
            }
        }
    }
    return true;
}
```

4. 主函数:

主函数中先实现本地网卡的寻找并列出。

```
// 查找设备列表
if (pcap_findalldevs(&all_devices, error_buffer) == -1) {
    cerr << "Error finding devices: " << error_buffer << endl;
    return -1;
}

// 列出所有设备并编号
pcap_if_t* device;
cout << "Available network devices:" << endl;
for (device = all_devices; device; device = device->next) {
    cout << ++device_count << ". " << device->name;
    if (device->description) {
        cout << " (" << device->description << ")";
    }
    cout << endl;
}

if (device_count == 0) {
    cerr << "No devices found!" << endl;
    return -1;
}

cout << "Select a device to capture packets (1-" << device_count << "): ";
cin >> device_choice;

if (device_choice < 1 || device_choice > device_count) {
    cerr << "Invalid choice!" << endl;
    pcap_freealldevs(all_devices);
    return -1;
}

// 找到用户选择的设备
device = all_devices;
for (int i = 1; i < device_choice; ++i) {
    device = device->next;
}
```

用户选择后，通过用户选择的网卡序号，利用 `pcap_open` 获取设备。

下面是构建 ARP 报文并进行发送和接收的过程。

```
BuildRequest(&eh, &ah, sendbuf, MyMac, MyIp, RevIp);
if (pcap_sendpacket(handle, sendbuf, 42) == 0)
    cout << "Send ARP request successfully." << endl;

if (!CapPacket(handle, SendIp, RevIp, MyIp, MyMac)) {
    cout << "Failed to capture ARP." << endl;
    pcap_freealldevs(all_devices);
    return -1;
}
```

实验结果

```
Microsoft Visual Studio 调试 × + ∨

Available network devices:
1. \Device\NPF_{DD3C0918-05AC-4D80-BA79-F6D16635202C} (VMware Virtual Ethernet Adapter)
2. \Device\NPF_{E8CCD4CB-9E78-47E4-A0FF-34F1B5B5142B} (VMware Virtual Ethernet Adapter)
3. \Device\NPF_{09BA378C-E304-4440-A17E-879D48135BEF} (Microsoft)
4. \Device\NPF_{013E0D5D-783A-4796-985A-59EDADD81DFC} (Microsoft)
5. \Device\NPF_{F40E2477-0DFF-4517-87E6-75E4A2BC1B39} ( )
Select a device to capture packets (1-5): 1
```

程序运行后，会出现一个查询本地网卡的界面，选择网卡序号，通过程序捕获的 ARP 报文可以观察得出，此时主机通过广播报文来查询对应本机的 IP 和 MAC 映射关系。

```
Microsoft Visual Studio 调试 × + ∨

Available network devices:
1. \Device\NPF_{DD3C0918-05AC-4D80-BA79-F6D16635202C} (VMware Virtual Ethernet Adapter)
2. \Device\NPF_{E8CCD4CB-9E78-47E4-A0FF-34F1B5B5142B} (VMware Virtual Ethernet Adapter)
3. \Device\NPF_{09BA378C-E304-4440-A17E-879D48135BEF} (Microsoft)
4. \Device\NPF_{013E0D5D-783A-4796-985A-59EDADD81DFC} (Microsoft)
5. \Device\NPF_{F40E2477-0DFF-4517-87E6-75E4A2BC1B39} ( )
Select a device to capture packets (1-5): 1
Open successful!
Get Host Ip:192.168.222.1
Send ARP request successfully.
ARP type:ARP request
Source Mac Address: 00:00:00:00:00:00
Source Ip Address: 0.0.0.0
Target Mac Address: FF:FF:FF:FF:FF:FF
Target Ip Address: 192.168.222.1
ARP type:ARP response
Source Mac Address: 00:50:56:C0:00:08
Source Ip Address: 192.168.222.1
Target Mac Address: 00:00:00:00:00:00
Target Ip Address: 0.0.0.0
Captured ARP packet with matching IP and MAC address:
IP: 192.168.222.1
MAC: 00:50:56:C0:00:08
```

通过此时 Wireshark 捕获到的数据包，也可以证实。

VMware Network Adapter VMnet8

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(I) 帮助(H)

arp

No.	Time	Source	Destination	Protocol	Length	Info
126	6.235919	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
159	7.235001	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
253	11.724554	VMware_92:d4:bd	VMware_e9:3a:8d	ARP	60	Who has 192.168.222.2? Tell 192.168.222.131
254	11.724592	VMware_e9:3a:8d	VMware_92:d4:bd	ARP	42	192.168.222.2 is at 00:50:56:e9:3a:8d
422	19.114394	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
435	19.741390	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
466	20.734051	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
496	22.125819	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
508	22.740570	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
515	23.741767	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
518	25.346072	00:00:00:00:00:00	Broadcast	ARP	42	Who has 192.168.222.1? Tell 0.0.0.0
519	25.346078	00:00:00:00:00:00	Broadcast	ARP	42	Who has 192.168.222.1? Tell 0.0.0.0
520	25.346088	VMware_c0:00:08	00:00:00:00:00:00	ARP	42	192.168.222.1 is at 00:50:56:c0:00:08
521	25.346090	VMware_c0:00:08	00:00:00:00:00:00	ARP	42	192.168.222.1 is at 00:50:56:c0:00:08
522	28.160408	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
523	28.160416	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
524	28.737415	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
525	28.737434	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
526	29.731314	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
527	29.731326	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
528	31.182186	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
529	31.182206	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
530	31.743058	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
531	31.743065	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
548	32.071565	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.131? Tell 192.168.222.1
549	32.071571	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.131? Tell 192.168.222.1
550	32.071855	VMware_92:d4:bd	VMware_c0:00:08	ARP	60	192.168.222.131 is at 00:0c:29:92:d4:bd
551	32.740804	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1
552	32.740813	VMware_c0:00:08	Broadcast	ARP	42	Who has 192.168.222.2? Tell 192.168.222.1

> Frame 42: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{DD3C0918-05AC-4D80-BA79-F6D16635202C}

> Ethernet II, Src: VMware_c0:00:08 (00:50:56:c0:00:08), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

> Address Resolution Protocol (request)

Address Resolution Protocol: Protocol

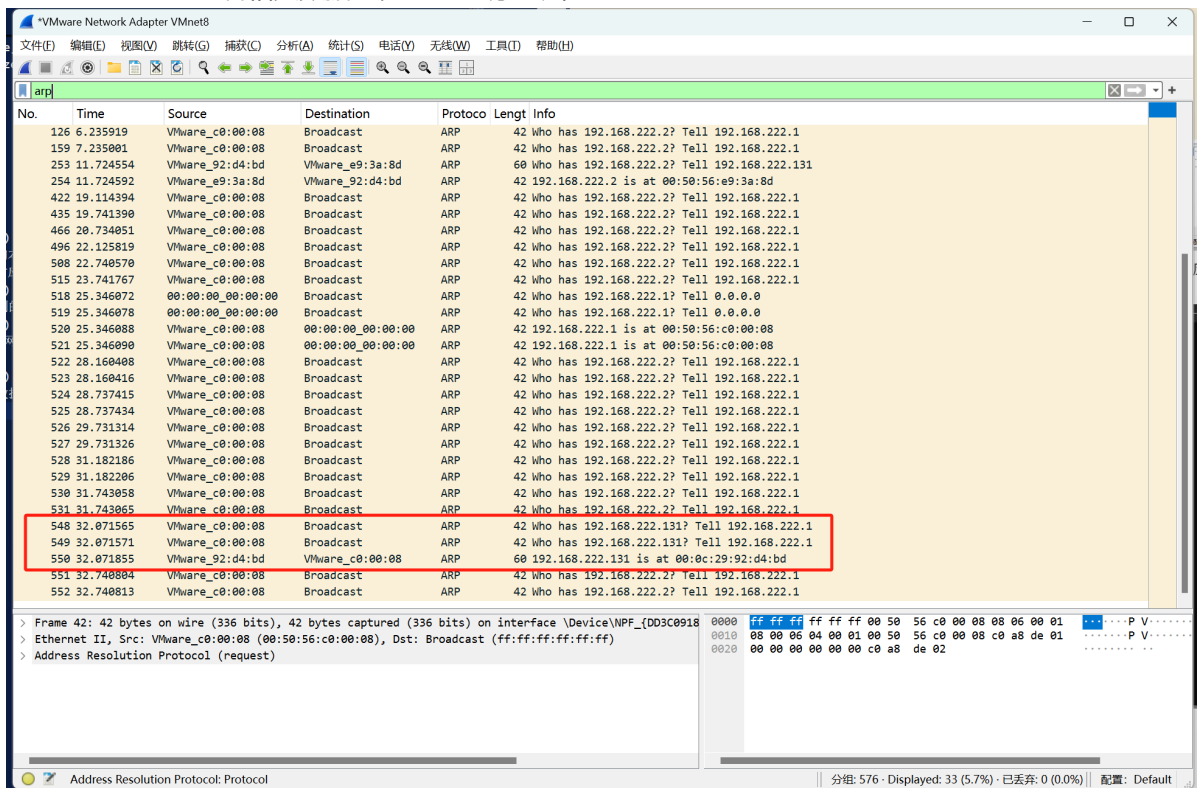
分组: 576 - Displayed: 33 (5.7%) - 已丢弃: 0 (0.0%) 配置: Default

输入目标 IP，此时主机在子网中发送广播地址寻找我输入的目标 IP，目标 IP 接收到请求后发送响应报文。

```
Microsoft Visual Studio 调试 X + -
Target Ip Address:192.168.222.131
Input RevIp:192.168.222.131
Send ARP successfully.
ARP type:ARP request
Source Mac Address: 00:50:56:C0:00:08
Source Ip Address: 192.168.222.1
Target Mac Address: 00:00:00:00:00:00
Target Ip Address: 192.168.222.2
ARP type:ARP request
Source Mac Address: 00:50:56:C0:00:08
Source Ip Address: 192.168.222.1
Target Mac Address: 00:00:00:00:00:00
Target Ip Address: 192.168.222.2
ARP type:ARP request
Source Mac Address: 00:50:56:C0:00:08
Source Ip Address: 192.168.222.1
Target Mac Address: 00:00:00:00:00:00
Target Ip Address: 192.168.222.2
ARP type:ARP request
Source Mac Address: 00:50:56:C0:00:08
Source Ip Address: 192.168.222.1
Target Mac Address: 00:00:00:00:00:00
Target Ip Address: 192.168.222.2
ARP type:ARP request
Source Mac Address: 00:50:56:C0:00:08
Source Ip Address: 192.168.222.1
Target Mac Address: FF:FF:FF:FF:FF:FF
Target Ip Address: 192.168.222.131
ARP type:ARP response
Source Mac Address: 00:0C:29:92:D4:BD
Source Ip Address: 192.168.222.131
Target Mac Address: 00:50:56:C0:00:08
Target Ip Address: 192.168.222.1
Captured ARP packet with matching IP and MAC address:
IP: 192.168.222.131
MAC: 00:0C:29:92:D4:BD

D:\Desktop\nt-lab3\lab3\x64\Debug\lab3.exe (进程 26060)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . . .]
```

通过 Wireshark 查看相关数据包, 也可证明正确性。



实验反思

本次实验是通过自己构建 ARP 报文, 利用实验一中的捕获数据包的方式, 抓取筛选其中的 ARP 请求及响应数据包, 判断条件是当一个 ARP 请求报文中的源 IP 地址和响应报文中的目的 IP 地址相同、响应报文中的源 IP 地址和请求报文中的目的 IP 地址相同时, 及获取了一个完整的 ARP 过程, 输出其中的 IP 及 MAC 地址。

在实验过程中, 我首先遇到的问题是 MAC 地址用 cout 输出时会出现乱码情况, 所以在比较后我选用 printf 来实现。其次, 在实验过程中我发现在主机发送请求后, 只有网关在和主机响应。后来通过咨询助教, 我了解到这个过程需要主机与目的靶机在同一个子网内, 不在同一个子网的目的靶机不会收到主机发送的 ARP 请求。如图:

```
D:\Desktop\lab3\lab3\64\ x + v
3. \Device\NPF_{09BA378C-E304-4440-A17E-879D48135BEF} (Microsoft)
4. \Device\NPF_{013E0D5D-783A-4796-985A-59EDADD81DFC} (Microsoft)
5. \Device\NPF_{F40E2477-0DFF-4517-87E6-75E4A2BC1B39} ( )
6. \Device\NPF_{49C9FFD6-A65C-45E4-87DC-512B5379BB7} (Microsoft)
Select a device to capture packets (1-6): 2
Open successful!
Get Host Ip:192.168.38.1
Send ARP request successfully.
ARP type:ARP request
Source Mac Address: 00:00:00:00:00:00
Source Ip Address: 0.0.0.0
Target Mac Address: FF:FF:FF:FF:FF:FF
Target Ip Address: 192.168.38.1
ARP type:ARP response
Source Mac Address: 00:50:56:C0:00:01
Source Ip Address: 192.168.38.1
Target Mac Address: 00:00:00:00:00:00
Target Ip Address: 0.0.0.0
Captured ARP packet with matching IP and MAC address:
IP: 192.168.38.1
MAC: 00:50:56:C0:00:01
Target Ip Address:192.168.222.131
Input RevIp:192.168.222.131
Send ARP successfully.
ARP type:ARP request
Source Mac Address: 00:50:56:C0:00:01
Source Ip Address: 192.168.38.1
Target Mac Address: FF:FF:FF:FF:FF:FF
Target Ip Address: 192.168.222.131
```

这个情况为网卡的 IP 地址为 192.168.38.1，但是我想获取的目的 IP 为 192.168.222.131，在同为子网掩码为 255.255.255.0 的情况下，这两个 IP 不在同一个子网中，所以会出现发送请求后目的靶机没有响应的情况。通过 Wireshark 进行抓包也可以看出来。

正在捕获 VMware Network Adapter VMnet1

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

arp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	00:00:00_00:00:00:00	Broadcast	ARP	42	Who has 192.168.38.1? Tell 0.0.0.0
2	0.000008	00:00:00_00:00:00:00	Broadcast	ARP	42	Who has 192.168.38.1? Tell 0.0.0.0
3	0.000023	VMware_c0:00:01	00:00:00_00:00:00:00	ARP	42	192.168.38.1 is at 00:50:56:c0:00:01
4	0.000027	VMware_c0:00:01	00:00:00_00:00:00:00	ARP	42	192.168.38.1 is at 00:50:56:c0:00:01
5	7.658958	VMware_c0:00:01	Broadcast	ARP	42	Who has 192.168.222.131? Tell 192.168.38.1
6	7.658965	VMware_c0:00:01	Broadcast	ARP	42	Who has 192.168.222.131? Tell 192.168.38.1

VMware Network Adapter VMnet1: <live capture in progress>

分组: 10 · Displayed: 6 (60.0%)

配置: Default